

Data Leakage e Scelta delle Foto per un Dataset

Applicazione al riconoscimento delle carte napoletane

Abstract

Questo documento riassume in modo dettagliato le problematiche di **data leakage** (fuga di informazioni) nel machine learning, basandosi sul lavoro di Apicella, Isgrò e Prevete (2025). Le informazioni sono contestualizzate per la costruzione di un dataset per il riconoscimento delle **carte napoletane**, organizzato in due varianti:

- **Dataset a 40 classi:** una classe per ciascuna carta (es. Asso di Coppe, Due di Bastoni, ...).
- **Dataset multi-annotato a 14 classi:** 10 classi per i valori (Asso, 2, 3, ..., 7, Fante, Cavallo, Re) e 4 classi per i semi (Coppe, Denari, Bastoni, Spade).

Entrambi i dataset sono suddivisi in tre split: **train**, **valid** e **test**.

Contents

1 Cos'è il Data Leakage

Il **data leakage** (letteralmente “fuga di dati”) si verifica quando informazioni che *non dovrebbero essere disponibili* durante l’addestramento di un modello di machine learning vengono inavvertitamente utilizzate, alterando la valutazione delle prestazioni. In pratica, il modello sembra funzionare molto meglio di quanto non faccia realmente su dati nuovi e mai visti.

Il data leakage porta a **stime eccessivamente ottimistiche** delle prestazioni che non si mantengono in scenari reali. La discrepanza tra prestazioni valutate e prestazioni effettive su nuovi dati è un problema critico.

Esempio applicato al dataset di carte napoletane

Se nel dataset di carte napoletane il modello raggiunge il 99% di accuratezza in validazione ma poi fallisce miseramente quando gli si mostra una carta fotografata in condizioni diverse, è probabile che ci sia stato un data leakage durante la costruzione del dataset o l’addestramento.

2 Tassonomia del Data Leakage

Il data leakage può verificarsi in **tre momenti critici** della pipeline di machine learning:

1. **Durante la definizione del dataset** — problematiche legate a come i dati vengono raccolti o generati (*Data-induced leakage*).
2. **Prima della separazione train/test** — il preprocessing dei dati può introdurre involontariamente leakage (*Preprocessing-related leakage*).
3. **Durante lo split dei dati** — il modo in cui i dati vengono suddivisi può causare leakage (*Split-related leakage*).

Nei prossimi paragrafi ciascuna categoria viene analizzata in dettaglio.

3 Data-Induced Leakage (Leakage indotto dai dati)

Questo tipo di leakage si origina **durante la costruzione stessa del dataset**. Le feature che compongono l’input della pipeline ML contengono informazioni “spurie” artificiali che guidano il modello verso la risposta corretta non grazie alle caratteristiche reali del compito, ma grazie a caratteristiche specifiche e non intenzionali del dataset adottato.

Si distinguono tre sottocategorie.

3.1 Data Collecting Leakage (Leakage da raccolta dati)

Si verifica quando il **processo di acquisizione** dei dati introduce informazioni spurie o correlazioni non intenzionali.

3.1.1 Leakage dovuto alle fonti di acquisizione

Quando i dati vengono raccolti da più soggetti o fonti con livelli di esperienza diversi, e non si tiene conto di queste differenze, il modello può adattarsi a pattern specifici di particolari fonti piuttosto che imparare le caratteristiche generali del compito.

Esempio applicato al dataset di carte napoletane

Se le foto delle carte napoletane vengono scattate da 5 persone diverse, e ogni persona fotografa solo alcune carte, il modello potrebbe imparare a riconoscere lo *stile fotografico* del fotografo (illuminazione, angolazione, sfondo) piuttosto che la carta stessa. Se poi training e test contengono foto degli stessi fotografi, le prestazioni risulteranno gonfiate.

Rischio concreto: il fotografo A scatta tutte le Coppe con luce calda e sfondo legno; il fotografo B scatta tutti i Bastoni con luce fredda e sfondo bianco. Il modello impara a distinguere luce/sfondo, non le carte.

3.1.2 Leakage dovuto al design dell'acquisizione

Il modo in cui l'esperimento di raccolta dati è progettato può introdurre artefatti. Ad esempio, se tutti gli stimoli di una stessa classe vengono presentati consecutivamente ("in blocco"), il modello potrebbe sfruttare artefatti temporali piuttosto che le caratteristiche reali dei dati.

Esempio applicato al dataset di carte napoletane

Se durante la sessione fotografica si fotografano prima tutte le 10 carte di Coppe, poi tutte le 10 di Denari, ecc., le condizioni ambientali (luce che cambia durante il giorno, stanchezza del fotografo, variazione dello sfondo) creano correlazioni spurie tra le foto dello stesso blocco. Il modello potrebbe sfruttare queste variazioni temporali piuttosto che le caratteristiche delle carte.

Soluzione: randomizzare l'ordine di acquisizione delle carte.

3.1.3 Leakage dovuto all'aggregazione dei dati (Dataset Frankenstein)

Quando un dataset è composto aggregando più dataset esistenti (**dataset Frankenstein**), può accadere che gli stessi dati siano presenti in più sotto-dataset. Se non si controlla la composizione, dati identici o sovrapposti possono finire sia nel training che nel test set.

Attenzione

I dataset Frankenstein sono particolarmente insidiosi perché la sovrapposizione non è sempre evidente. Due sotto-dataset D_{s1} e D_{s2} possono condividere istanze ($D_{s1} \cap D_{s2} \neq \emptyset$) senza che ce ne si accorga.

Esempio applicato al dataset di carte napoletane

Se si combinano foto di carte napoletane scaricate da fonti diverse su internet (es. immagini da Google, da altri dataset pubblici, da app di gioco), alcune immagini potrebbero essere identiche o molto simili, finendo sia nel train che nel test. Il modello "ricorda" quelle immagini e le classifica correttamente, gonfiando le

prestazioni.

Soluzione: scattare personalmente tutte le foto, oppure verificare attentamente l'unicità di ogni immagine.

3.2 Label Leakage (Leakage dalle etichette)

Si verifica quando l'etichetta (la classe corretta) è presente, direttamente o indirettamente, nelle feature usate come input del modello.

3.2.1 Label Leakage diretto

L'etichetta reale è **direttamente contenuta** nelle feature dei dati. In termini formali:

$$\exists i \in \{1, \dots, |D|\}, f \in \{1, \dots, d\} \quad \text{t.c.} \quad x_f^{(i)} \leftarrow y^{(i)}$$

Il modello sfrutta la feature coincidente con l'etichetta, ottenendo prestazioni artificialmente elevate. Inoltre, su dati nuovi privi di questa feature, il modello risulta inutilizzabile.

Esempio applicato al dataset di carte napoletane

Se nel nome del file di ogni immagine è presente il nome della carta (es. `asso_di_coppe_001.jpg`) e questo nome viene in qualche modo passato come feature al modello (ad esempio tramite metadati EXIF o path del file), il modello può “leggere” direttamente la risposta corretta.

Soluzione: assicurarsi che i nomi dei file e i metadati non contengano informazioni sulla classe. Usare identificatori anonimi.

Chiarimento importante: nomi dei file ed etichette

Il modello di machine learning **non si addestra dal nome del file**. In fase di addestramento, il modello riceve come input solo i **pixel dell'immagine** e come output atteso l'**etichetta** (la classe). Il nome del file non viene mai passato direttamente alla rete neurale.

È perfettamente lecito usare i nomi dei file per recuperare le classi per comodità durante la *preparazione* del dataset (es. dal nome `asso_di_coppe_001.jpg` si estrae l'etichetta “asso di coppe”). Il problema si pone **solo** in situazioni particolari in cui il nome o il path del file viene inavvertitamente incluso come feature di input (ad esempio come stringa testuale in un modello multi-modale, oppure se metadati EXIF incorporano il nome della classe).

In sintesi, il flusso corretto è:

1. **Preparare le etichette una volta sola:** usare i nomi originali dei file per assegnare le classi (es. parsing di `asso_di_coppe` → classe 0). Si può creare un file CSV separato con il mapping `filename` → `etichetta`.
2. **Rinominare i file con ID anonimi** (opzionale ma consigliato): sostituire i nomi con identificatori neutri (es. `img_0001.jpg`, `img_0002.jpg`) tramite uno script automatico. Questo elimina qualsiasi rischio residuo.
3. **Pulire i metadati:** rimuovere le informazioni EXIF o altri metadati che potrebbero rivelare la classe (es. con il comando `exiftool -all= *.jpg`).
4. **Caricare i dati correttamente:** in fase di addestramento, fornire al modello

solo l'immagine (array di pixel) come input. L'etichetta viene letta dal file CSV o dalla struttura delle cartelle, ma **non viene mai passata come feature** al modello.

5. **Splittare dopo l'anonimizzazione:** dividere in train/valid/test solo dopo aver rinominato i file e pulito i metadati.
6. **Verificare:** testare il modello assicurandosi che la rete stia imparando dalle caratteristiche visive della carta e non da artefatti nascosti. Se l'accuratezza è sospettamente alta fin dalle prime epoche, potrebbe esserci un leakage.

In altre parole: **i nomi dei file servono a te** (il programmatore) per organizzare il dataset e assegnare le etichette, ma **non devono mai raggiungere il modello** come dato di input.

3.2.2 Label Leakage indiretto

Informazioni riguardanti le etichette corrette sono integrate nelle feature in modo **indiretto**, ad esempio come feature proxy calcolate a partire dall'etichetta. Formalmente:

$$\exists i, f_1, f_2, \dots, f_t \in \{1, \dots, d\}, k: \mathbb{R} \rightarrow \mathbb{R}^t \quad \text{t.c.} \quad x_{f_1, f_2, \dots, f_t}^{(i)} \leftarrow k(y^{(i)})$$

Leakage da feature spurie. Una feature apparentemente irrilevante (come un identificativo) può essere correlata con l'etichetta per ragioni indirette (es. i pazienti con ID basso appartengono prevalentemente a una classe). Questo è un caso del cosiddetto **effetto Clever Hans**: il modello impara la risposta giusta per le ragioni sbagliate.

Esempio applicato al dataset di carte napoletane

Se le foto delle carte sono organizzate in cartelle numerate progressivamente e il modello ha accesso all'indice progressivo, potrebbe scoprire che le prime 50 foto sono tutte Coppe, le successive 50 sono Denari, ecc. L'ordine di acquisizione diventa una feature spuria correlata con la classe.

Altro esempio: se tutte le foto dell'Asso di Spade hanno una risoluzione di 1920×1080 mentre quelle del Re di Coppe hanno risoluzione 800×600 , la risoluzione diventa un proxy dell'etichetta.

Buona pratica

Per individuare l'effetto Clever Hans si possono usare metodi di **Explainable AI (XAI)** come LIME o Grad-CAM, che mostrano su quali parti dell'immagine il modello si sta concentrando. Se il modello guarda lo sfondo invece della carta, c'è un problema.

3.3 Synthesis Leakage (Leakage da sintesi dei dati)

Si verifica quando una funzione di generazione $g: \mathcal{D} \rightarrow \mathcal{D}$ viene applicata ai dati **prima dello split**, e dati nel training e nel test vengono generati a partire dagli stessi dati originali. Questo riguarda tipicamente le tecniche di **oversampling** e **undersampling**.

3.3.1 Leakage da oversampling

Quando si usa SMOTE o tecniche simili per bilanciare le classi **prima** di dividere i dati, i campioni sintetici generati potrebbero essere troppo simili a campioni che finiranno nel test set (poiché i vicini usati per la generazione possono essere nel test). Inoltre, dati sintetici possono erroneamente finire nel set di valutazione.

3.3.2 Leakage da undersampling

Analogamente, metodi come Cluster Centroids calcolano centroidi usando dati che poi finiscono nel test set, introducendo informazioni non lecite nel training.

Attenzione

Le operazioni di oversampling e undersampling devono essere eseguite **esclusivamente dopo** lo split dei dati, e **solo** sul training set. Mai sui dati di validazione o test.

Esempio applicato al dataset di carte napoletane

Se nel dataset a 40 classi alcune carte hanno poche foto (es. solo 15 foto del 7 di Bastoni) e si decide di usare data augmentation o oversampling per bilanciare:

- **Sbagliato:** applicare augmentation sull'intero dataset e poi fare lo split → le immagini aumentate derivate da foto del test possono finire nel train.
- **Corretto:** prima dividere in train/valid/test, poi applicare augmentation **solo** sul train set.

4 Preprocessing-Related Leakage (Leakage da preprocessing)

Questo tipo di leakage si verifica quando le procedure di preprocessing vengono applicate **prima** di una corretta separazione tra dati di training e dati di valutazione/test. Queste procedure richiedono parametri (es. range delle feature, statistiche) che vengono stimati dai dati. Se la stima include dati del test set, si ha leakage.

Formalmente, data una funzione di preprocessing $f : \mathcal{D} \times Q \rightarrow \mathcal{D}$ e una funzione $v : \mathcal{D} \rightarrow Q$ che stima i parametri dai dati: se $Q = v(D')$ con $D' \supseteq D_E$, allora training e evaluation condividono informazioni, introducendo leakage.

4.1 Normalization Leakage

Si verifica quando una funzione di normalizzazione è applicata usando parametri calcolati sull'**intero dataset** invece che solo sul training set.

Esempio con min-max scaling. Se le feature vengono normalizzate nell'intervallo $[0, 1]$ usando i valori minimo e massimo calcolati su *tutto* il dataset D , e poi si fa lo split, i dati di training incorporano informazione sulla distribuzione dei dati di test (attraverso i valori min e max).

Esempio con standardizzazione. Se media μ e deviazione standard σ sono calcolate su tutto D , la media risultante è significativamente diversa dalla media calcolata solo su D_T . Usare la media dell'intero dataset per standardizzare fornisce informazioni sui dati di test ai dati di training.

Esempio applicato al dataset di carte napoletane

Se le immagini delle carte vengono normalizzate (es. standardizzazione dei valori dei pixel) calcolando media e deviazione standard su *tutte* le immagini (train + valid + test):

- La media dei pixel calcolata includerà la distribuzione delle immagini di test.
- Il modello “conoscerà” indirettamente le caratteristiche statistiche del test set.

Soluzione: calcolare μ e σ **solo** sulle immagini di training, poi applicare la stessa trasformazione (con gli stessi parametri) a valid e test.

4.2 Cleaning Leakage

Si verifica quando tecniche di pulizia dei dati (es. media mobile per ridurre il rumore) vengono applicate sull'intero dataset, facendo sì che i dati di training incorporino informazioni dai dati di test. Ad esempio, se si usa una media mobile su dati sequenziali, l'ultimo punto del training set potrebbe usare il primo punto del test set per calcolare il suo valore smussato.

4.3 Imputation Leakage

Quando ci sono valori mancanti e si usa un algoritmo come KNN-Impute per stimarli, se l'imputazione viene fatta sull'intero dataset, i valori stimati per i dati di training possono basarsi su vicini che si trovano nel test set (e viceversa).

4.4 Feature Engineering Leakage

Se si effettua una **feature selection** sull'intero dataset (es. selezionando le feature più discriminanti tramite un t-test), le feature scelte sono influenzate anche dai dati di valutazione/test. Il processo di selezione è “contaminato”.

Buona pratica

Ogni operazione di preprocessing, normalizzazione, pulizia, imputazione e feature engineering deve essere eseguita **dopo** lo split dei dati, e i parametri devono essere calcolati **esclusivamente** sul training set.

Schema corretto:

1. Split dei dati in train, valid e test.
2. Calcolo dei parametri di preprocessing solo sul train set.
3. Applicazione della trasformazione (con gli stessi parametri) a train, valid e test.

4.5 Caso particolare: leakage che colpisce il validation set

Un caso insidioso si verifica quando il training set D_T viene ulteriormente diviso in training effettivo D'_T e validation set D_V (usato per early stopping), **ma il preprocessing è stato fatto su tutto D_T** prima di questa suddivisione. In questo caso:

- D_V contiene informazioni provenienti da D'_T (e viceversa) attraverso i parametri di preprocessing.
- Il criterio di early stopping basato su D_V potrebbe non riflettere la vera capacità di generalizzazione del modello.
- Il modello selezionato potrebbe non essere il migliore per dati realmente nuovi (D_E), portando a overfitting o under-training.

Buona pratica

La suddivisione corretta prevede:

1. Split in train, valid e test.
2. Calcolo parametri di preprocessing **solo** su train.
3. Applicazione a train, valid e test con gli stessi parametri.
4. Training con early stopping monitorato sul valid set.

5 Split-Related Leakage (Leakage da strategia di split)

Anche quando i dati sono stati preparati correttamente, il **modo in cui vengono suddivisi** in train, valid e test può introdurre leakage. Si distinguono due categorie principali.

5.1 Similarity Leakage

Training e test contengono dati uguali, simili o con porzioni in comune.

5.1.1 Sets' Intersection Leakage (intersezione tra insiemi)

Si verifica quando la strategia di split assegna gli **stessi dati** sia a D_T che a D_E . Può accadere con:

- Procedure di campionamento **con rimpiazzamento**.
- Uso del test set come validation set per early stopping ($D_V = D_E$), il che fa sì che il punto di arresto del training sia influenzato da dati che il modello non dovrebbe conoscere.

Anche un **basso livello di contaminazione** del training set con dati del test porta a stime gonfiate delle prestazioni.

Esempio applicato al dataset di carte napoletane

Se la stessa identica foto di una carta napoletana finisce sia nel train che nel test (per errore di duplicazione o perché si è usato un campionamento con rimpiazzamento), il modello la “memorizza” e la classifica correttamente nel test senza aver davvero imparato.

Rischio concreto con le carte: se si scattano più foto molto simili della stessa carta (stessa posizione, stessa luce, stessa angolazione) e alcune finiscono nel train e altre nel test, il modello può “ricordare” quella specifica foto piuttosto che imparare le caratteristiche della carta.

5.1.2 Data Overlap Leakage (sovrapposizione dei dati)

Si verifica quando le **feature di istanze diverse si sovrappongono** tra training e test. In termini formali:

$$\exists (x^{(i)}, y^{(i)}) \in D_T, \exists (x^{(l)}, y^{(l)}) \in D_E \quad \text{t.c. porzioni di } x^{(i)} \text{ e } x^{(l)} \text{ sono condivise}$$

Questo avviene tipicamente quando i dati vengono segmentati in finestre sovrapposte (es. segnali EEG con overlap del 50%): finestre consecutive condividono metà dei dati, e se finiscono in set diversi, il modello ha già “visto” parte del test.

Esempio applicato al dataset di carte napoletane

Se si estraggono più crop (ritagli) dalla stessa immagine di una carta e questi crop finiscono in split diversi, il modello sfrutta le parti sovrapposte. Analogamente, se si applicano trasformazioni minime (es. rotazione di 1) e la versione originale è nel train mentre quella ruotata è nel test, si ha overlap.

Regola: ogni **foto originale** deve appartenere a **un solo split**. Tutte le augmentation derivate dalla stessa foto devono stare nello stesso split.

5.2 Distribution Leakage

Si verifica quando lo split **non tiene conto delle diverse distribuzioni** dei dati, introducendo conoscenza nel training non rappresentativa del compito reale. Formalmente, se i dati provengono da molteplici distribuzioni $P_1, P_2, \dots, P_k$ e lo split mescola le distribuzioni senza criterio, il training set e il test set condividono bias specifici di certe fonti.

5.2.1 Leakage da distribuzione condivisa tra istanze

Quando dati provenienti dalla **stessa fonte/soggetto** finiscono sia nel training che nel test, il modello può sfruttare artefatti o bias specifici di quella fonte.

Esempio applicato al dataset di carte napoletane

Consideriamo che le foto delle 40 carte napoletane siano scattate usando 3 mazzi fisici diversi (con usura, colori e qualità di stampa differenti):

- **Mazzo A:** carte nuove, colori vivaci.
- **Mazzo B:** carte usate, colori sbiaditi.
- **Mazzo C:** carte di un produttore diverso.

Split sbagliato: mescolare casualmente tutte le foto → nel test ci sono foto del Mazzo A, e nel train ci sono *altre* foto dello stesso Mazzo A. Il modello impara le caratteristiche specifiche del mazzo (texture, colore della stampa) e “generalizza” solo perché riconosce il mazzo, non la carta.

Split corretto: considerare ogni mazzo come una distribuzione separata. Se si vuole testare la generalizzazione a mazzi diversi, il test set dovrebbe contenere foto di un mazzo *non presente* nel training.

Se invece l’obiettivo è il riconoscimento su mazzi noti, lo split casuale è accettabile, ma deve essere **esplicitamente dichiarato**.

5.2.2 Leakage da dipendenza temporale

Quando i dati hanno una relazione sequenziale nel tempo, mescolarli casualmente tra train e test può far sì che il modello “impari dal futuro”. Dati futuri possono contenere informazioni che aiutano a inferire dati passati.

Esempio applicato al dataset di carte napoletane

Se le foto vengono scattate in sessioni diverse (mattina e pomeriggio) e la luce cambia progressivamente, foto della sessione pomeridiana (nel train) contengono informazioni sulla “direzione” del cambiamento di luce che aiutano a classificare le foto della sessione mattutina (nel test). In generale, per un dataset di carte questo rischio è minore, ma occorre comunque randomizzare le condizioni di acquisizione.

6 Informazioni spurie e concetto di Spurious Feature

Una **feature spuria** è un’informazione contenente pattern correlati con l’etichetta del compito ma **non intrinsecamente rilevanti** per il compito stesso. La proprietà di essere spuria dipende **fortemente dal compito**.

Esempio applicato al dataset di carte napoletane

Nel dataset di carte napoletane:

- Lo **sfondo della foto** è una feature spuria se il compito è riconoscere la carta (il modello non dovrebbe guardare lo sfondo).
- La **qualità dell’illuminazione** è una feature spuria.
- La **mano** che tiene la carta è una feature spuria.
- La **superficie del tavolo** su cui è appoggiata è una feature spuria.

Tuttavia, se il compito fosse “distinguere in quale stanza è stata fotografata la carta”, lo sfondo diventerebbe rilevante e non più spurio.

7 Linee guida pratiche per il dataset di carte napoletane

Basandosi su tutte le problematiche analizzate, ecco le linee guida operative.

7.1 Scelta e acquisizione delle foto

- R1. Randomizzare l'ordine di acquisizione:** non fotografare tutte le carte di uno stesso seme in sequenza. Mescolare l'ordine.
- R2. Variare le condizioni:** per ogni carta, variare sistematicamente sfondo, illuminazione, angolazione, distanza. Questo riduce le feature spurie.
- R3. Controllare la consistenza:** assicurarsi che la variabilità delle condizioni sia *uniforme* tra tutte le classi. Ogni carta deve essere fotografata nelle stesse condizioni eterogenee.
- R4. Usare più mazzi fisici:** per aumentare la generalizzazione, usare carte di produttori diversi, con livelli di usura diversi.
- R5. Evitare dataset Frankenstein:** se si integrano foto da fonti esterne, verificare attentamente che non ci siano duplicati.
- R6. Anonimizzare i metadati:** rimuovere informazioni EXIF, usare nomi di file non informativi (es. `img_00001.jpg` con mappatura separata in un file CSV).
- R7. Non includere feature proxy:** nessuna informazione accessoria (posizione nel filesystem, ordine di acquisizione, dimensione del file) deve essere correlata con la classe.

7.2 Strategia di split train/valid/test

- S1. Ogni foto originale in un solo split:** se la foto X è nel training, *tutte* le augmentation di X devono stare nel training. Mai split diversi per la stessa foto o sue versioni derivate.
- S2. Nessuna intersezione:** $D_{\text{train}} \cap D_{\text{valid}} = \emptyset$, $D_{\text{train}} \cap D_{\text{test}} = \emptyset$, $D_{\text{valid}} \cap D_{\text{test}} = \emptyset$.
- S3. Split bilanciato:** ogni split deve avere una distribuzione simile delle classi (stratificazione).
- S4. Considerare le distribuzioni:** se si usano più mazzi, decidere *a priori* se il compito richiede generalizzazione a mazzi nuovi (split per mazzo) o solo a nuove foto degli stessi mazzi (split casuale stratificato).
- S5. Split prima del preprocessing:** dividere i dati *prima* di qualsiasi normalizzazione, augmentation o bilanciamento.
- S6. Non usare il test set per early stopping:** il validation set deve essere separato dal test set. Il test set va usato **una sola volta** alla fine.
- S7. Per il dataset multi-annotato (14 classi):** lo split deve garantire il bilanciamento sia per i 10 valori che per i 4 semi, dato che le annotazioni sono multiple.

7.3 Preprocessing corretto

- P1. Prima:** split in train/valid/test.
- P2. Poi:** calcolare i parametri di normalizzazione (media, std dei pixel, ecc.) **solo** sul train set.
- P3. Infine:** applicare la stessa trasformazione a train, valid e test usando i parametri calcolati sul train.
- P4. Augmentation solo su train:** data augmentation (rotazioni, flip, variazioni di colore, ecc.) va applicata esclusivamente ai dati di training.
- P5. Oversampling/undersampling solo su train:** se alcune classi sono sbilanciate, il bilanciamento va fatto solo dopo lo split e solo sul train.

8 Riepilogo schematico

Tipo di leakage	Come si manifesta	Come prevenirlo
Data collecting	Condizioni sistematiche diverse per classi diverse; dataset Frankenstein con duplicati	Randomizzare acquisizione; verificare unicità delle foto; evitare aggregazioni non controllate
Label leakage	Metadati o nomi file che rivelano la classe; feature correlate artificialmente con l'etichetta	Anonimizzare metadati; controllare correlazioni spurie; usare XAI
Synthesis leakage	Augmentation/oversampling prima dello split	Augmentation e bilanciamento solo dopo lo split e solo su train
Normalization leakage	Parametri (media, std, min, max) calcolati su tutto il dataset	Calcolare parametri solo su train
Feature engineering leakage	Feature selection su tutto il dataset	Feature selection solo su train
Sets' intersection	Stessi dati in train e test	Split senza rimpiazzamento; ogni foto in un solo split
Data overlap	Crop/augmentation della stessa foto in split diversi	Tutte le derivazioni di una foto nello stesso split
Distribution leakage	Dati dello stesso mazzo/fonte in train e test senza controllo	Split consapevole delle fonti; dichiarare il tipo di generalizzazione desiderata

Table 1: Riepilogo dei tipi di data leakage e relative strategie di prevenzione per il dataset di carte napoletane.

9 Conclusioni

La costruzione di un dataset per il riconoscimento delle carte napoletane richiede attenzione a **ogni fase** della pipeline: dalla raccolta delle foto, al preprocessing, fino allo split in train/valid/test. I principi fondamentali sono:

1. **Definire chiaramente il compito:** il dataset a 40 classi richiede che il modello distingua ogni singola carta; il dataset a 14 classi richiede il riconoscimento separato di valore e seme. La definizione del compito determina cosa costituisce leakage e cosa no.
2. **Separare prima, trasformare dopo:** qualsiasi operazione sui dati (normalizzazione, augmentation, bilanciamento) deve avvenire *dopo* lo split, con parametri calcolati solo sul training set.
3. **Garantire indipendenza reale tra gli split:** non basta che le immagini siano formalmente diverse; devono essere anche *statisticamente indipendenti* (niente crop sovrapposti, niente versioni derivate in split diversi).
4. **Controllare le feature spurie:** usare tecniche XAI per verificare che il modello stia effettivamente guardando le carte e non lo sfondo, l'illuminazione o altri artefatti.
5. **Documentare le scelte:** dichiarare esplicitamente la strategia di split, le condizioni di acquisizione, e il tipo di generalizzazione attesa.

Riferimento bibliografico: Apicella, A., Isgrò, F., & Prevete, R. (2025). *Don't push the button! Exploring data leakage risks in machine learning and transfer learning*. Artificial Intelligence Review, 58, 339. <https://doi.org/10.1007/s10462-025-11326-3>